



Aula 14: Manipulação de Strings em Java

Programação Orientada a Objetos

Nesta aula, vamos estudar como trabalhar com textos em Java, entendendo como comparar, manipular, dividir, substituir e validar Strings. Também veremos como aplicar esses recursos em situações comuns de sistemas orientados a objetos, como validação de nome, CPF, e-mail, senha e formatação de dados.

POO EM
JAVA

AULA 14

NÍVEL INICIANTE

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

01

Compreender o tipo String em Java

Entender que `String` é uma classe utilizada para representar textos e que possui métodos próprios para manipulação de caracteres.

03

Manipular textos com métodos da classe String

Utilizar métodos como `length()`, `charAt()`, `substring()`, `toUpperCase()`, `toLowerCase()`, `trim()`, `replace()` e `split()`.

02

Comparar Strings corretamente

Diferenciar o uso de `==` e `equals()`, compreendendo por que textos devem ser comparados pelo conteúdo e não pela referência de memória.

04

Aplicar Strings em validações de sistemas

Usar Strings para validar dados comuns em sistemas orientados a objetos, como nome, CPF, e-mail, senha, matrícula e placa de veículo.

O que é uma String em Java?

Em Java, `String` é usada para representar textos, como nomes, e-mails, senhas, códigos e mensagens.

Diferente de tipos como `int`, `double` e `boolean`, a `String` não é um tipo primitivo: ela é uma classe possuindo vários métodos prontos para manipular esse conteúdo.

```
String nome = "Maria";  
String cpf = "12345678900";  
String email = "maria@email.com";
```

Então com uma `String`, podemos descobrir o tamanho do texto, transformar letras, remover espaços, comparar conteúdos e separar informações.

```
System.out.println(nome.length());  
System.out.println(nome.toUpperCase());  
System.out.println(email.contains("@"));
```



Comparando Strings: erro comum

Um erro bastante comum em Java é comparar textos usando `==`.

```
String nome1 = "Maria";  
String nome2 = new String("Maria");  
System.out.println(nome1 == nome2); // false
```

Nesse caso, os dois textos possuem o mesmo conteúdo, mas estão em lugares diferentes da memória. O operador `==` verifica se as duas variáveis apontam para o mesmo objeto, e não se o texto escrito dentro delas é igual.

Para comparar o conteúdo de uma String, usamos `equals()`:

```
System.out.println(nome1.equals(nome2)); // true
```

Também podemos ignorar letras maiúsculas e minúsculas usando `equalsIgnoreCase()`:

```
String resposta = "SIM";  
if (resposta.equalsIgnoreCase("sim")) {  
    System.out.println("Resposta confirmada.");  
}
```



Métodos básicos de String

A classe `String` possui vários métodos prontos para consultar e transformar textos.

Função	Método
Retorna a quantidade de caracteres	<code>length();</code>
Remove espaços no início e no fim	<code>trim();</code>
Transforma o texto em maiúsculo	<code>toUpperCase();</code>
Transforma o texto em minúsculo	<code>toLowerCase();</code>
Retorna o caractere de uma posição	<code>charAt(0);</code>

```
String nome = " José ";

System.out.println(nome.length());      // 6
System.out.println(nome.trim());        // José
System.out.println(nome.toUppeQrCase()); //  José
System.out.println(nome.charAt(1));     // J
```

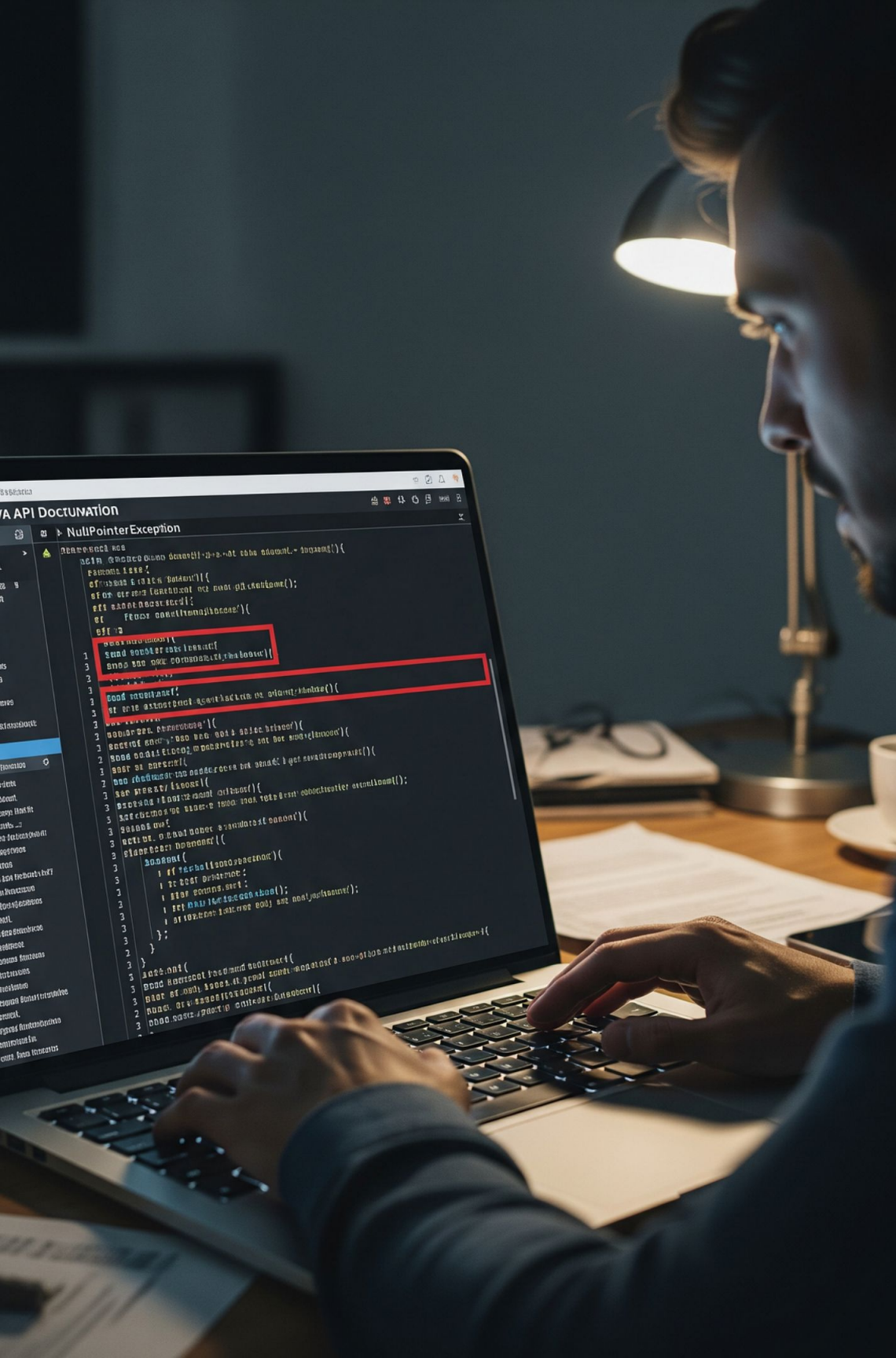
String é imutável

Métodos como `trim()` e `toUpperCase()` não alteram o texto original automaticamente.

```
String nome = " José ";  
  
nome.trim();  
  
System.out.println(nome); // continua " José "
```

Para guardar a alteração, precisamos atribuir o resultado:

```
nome = nome.trim();  
  
System.out.println(nome); // José
```



Métodos básicos de String

A classe String possui vários métodos prontos para consultar e transformar textos.

Função	Método
Retorna a quantidade de caracteres	length();
Remove espaços no início e no fim	trim();
Transforma o texto em maiúsculo	toUpperCase();
Transforma o texto em minúsculo	toLowerCase();
Retorna o caractere de uma posição	charAt(0);

Como String é imutável, métodos como `trim()` e `toUpperCase()` não alteram o texto original automaticamente.

```
String nome = " José ";  
nome.trim();  
System.out.println(nome); //continua " José "
```

Para guardar a alteração, precisamos atribuir o resultado:

```
nome = nome.trim();  
System.out.println(nome); //José
```

Pegando partes do texto

O método `substring()` permite extrair apenas uma parte de uma `String`. Ele é útil quando queremos separar informações de um texto maior, como código da disciplina, matrícula, placa ou data.

```
String codigo = "SIN141";

String sigla = codigo.substring(0, 3);
String numero = codigo.substring(3);

System.out.println(sigla); // SIN
System.out.println(numero); // 141
```

No exemplo:

```
codigo.substring(0, 3);
```

Ele pega os caracteres da posição 0 até antes da posição 3.

Verificando conteúdo

Além de comparar textos, também podemos verificar se uma `String` contém, começa ou termina com determinado valor. Esses métodos retornam `true` ou `false`.

Método	Função
<code>contains()</code>	Verifica se o texto contém uma parte específica
<code>startsWith()</code>	Verifica se o texto começa com determinado valor
<code>endsWith()</code>	Verifica se o texto termina com determinado valor

```
String email = "aluno@ufv.br";

System.out.println(email.contains("@"));      // true
System.out.println(email.startsWith("aluno")); // true
System.out.println(email.endsWith(".br"));    // true
```

Verificando conteúdo: Exemplo

```
public class VerificarTexto {  
    public static void main(String[] args) {  
        String email = "aluno@ufv.br";  
  
        if (email.contains("@")) {  
            System.out.println("O e-mail possui @");  
        }  
  
        if (email.startsWith("aluno")) {  
            System.out.println("O e-mail começa com aluno");  
        }  
  
        if (email.endsWith(".br")) {  
            System.out.println("O e-mail termina com .br");  
        }  
    }  
}  
  
// O e-mail possui @  
// O e-mail começa com aluno  
// O e-mail termina com .br
```

Substituindo caracteres

O método `replace()` permite substituir uma parte do texto por outra. Ele é muito usado para limpar dados, removendo caracteres que não serão armazenados no sistema.

```
String cpf = "123.456.789-00";

String cpfLimpo = cpf.replace(".", "").replace("-", "");

System.out.println(cpfLimpo); // 12345678900
```

Nesse exemplo, os pontos `.` e o traço `-` foram substituídos por vazio `""`. Ou seja, eles foram removidos da String. Também podemos usar para padronizar textos:

```
String telefone = "(34) 99999-8888";

telefone = telefone.replace("(", "")
                    .replace(")", "")
                    .replace("-", "")
                    .replace(" ", "");

System.out.println(telefone); // 34999998888
```

Dividindo texto com split

O método `split()` divide uma `String` em partes, usando um separador definido. Ele é muito útil quando recebemos dados em uma única linha, como cadastros, arquivos ou entradas digitadas pelo usuário.

```
String linha = "Maria;12345678900;22";

String[] partes = linha.split(";");

String nome = partes[0];
String cpf = partes[1];
String idade = partes[2];

System.out.println("Nome: " + nome);
System.out.println("CPF: " + cpf);
System.out.println("Idade: " + idade);

// Nome: Maria
// CPF: 12345678900
// Idade: 22
```


Validação de dados com String

Antes de armazenar um texto, podemos verificar se ele é válido.

```
public void setNome(String nome) {
    if (nome == null || nome.trim().isEmpty()) {
        throw new IllegalArgumentException("Nome não pode ser vazio.");
    }

    this.nome = nome.trim();
}
```

Trecho	Método
nome == null	Verifica se nenhum valor foi informado
trim()	Remove espaços no início e no fim
isEmpty()	Verifica se o texto está vazio
throw	Lança uma exceção caso o nome seja inválido

Validação de dados com String

Sem entrar em cálculo oficial de CPF, dá para validar formato básico:

```
public static boolean cpfValido(String cpf) {  
    if (cpf == null) {  
        return false;  
    }  
  
    cpf = cpf.replace(".", "").replace("-", "");  
  
    return cpf.length() == 11;  
}  
  
// Uso  
if (cpfValido("123.456.789-00")) {  
    System.out.println("CPF com formato válido");  
}
```

Exemplo em classe: Cliente

```
public class Cliente {  
    private String nome;  
    private String cpf;  
    private String email;  
  
    public Cliente(String nome, String cpf, String email) {  
        setNome(nome);  
        setCpf(cpf);  
        setEmail(email);  
    }  
  
    public void setNome(String nome) {  
        if (nome == null || nome.trim().isEmpty()) {  
            throw new IllegalArgumentException("Nome inválido.");  
        }  
  
        this.nome = nome.trim();  
    }  
}
```

Exemplo em classe: Cliente

```
public void setCpf(String cpf) {  
    cpf = cpf.replace(".", "").replace("-", "");  
  
    if (cpf.length() != 11) {  
        throw new IllegalArgumentException("CPF inválido.");  
    }  
  
    this.cpf = cpf;  
}  
  
public void setEmail(String email) {  
    if (email == null || !email.contains("@")) {  
        throw new IllegalArgumentException("E-mail inválido.");  
    }  
  
    this.email = email.trim().toLowerCase();  
}  
}
```

StringBuilder

Quando precisamos montar um texto grande aos poucos, podemos usar StringBuilder. Ele é útil para gerar relatórios, mensagens e saídas formatadas.

```
public class ExemploStringBuilder {  
    public static void main(String[] args) {  
        StringBuilder relatorio = new StringBuilder();  
  
        relatorio.append("Relatório de Clientes\n");  
        relatorio.append("-----\n");  
        relatorio.append("Nome: Maria\n");  
        relatorio.append("CPF: 12345678900\n");  
        relatorio.append("E-mail: maria@email.com\n");  
  
        System.out.println(relatorio.toString());  
    }  
}
```

📌 **StringBuilder** é recomendado quando o texto será construído ou alterado várias vezes dentro do programa.

Exemplo: Relatório de Clientes

```
public String gerarRelatorio() {
    StringBuilder sb = new StringBuilder();

    sb.append("Cliente\n");
    sb.append("-----\n");
    sb.append("Nome: ").append(nome).append("\n");
    sb.append("CPF: ").append(cpf).append("\n");
    sb.append("E-mail: ").append(email).append("\n");

    return sb.toString();
}

Cliente c = new Cliente("Maria", "12345678900", "maria@email.com");

System.out.println(c.gerarRelatorio());
```


Exercícios de fixação

Coloque em prática tudo o que foi aprendido nesta aula. A melhor forma de fixar conceitos de POO é escrever código — não apenas ler!

01

01 - Comparação de Strings

Explique por que o código abaixo pode dar resultado inesperado:

```
String a = new String("Java");  
String b = new String("Java");  
  
System.out.println(a == b);  
System.out.println(a.equals(b));
```

02

02 - Limpeza de CPF

Crie um método que receba um CPF com pontos e traço e retorne apenas os números.
Entrada: "123.456.789-00"
Saída: "12345678900"

03

03 - Separando dados

Dada a String: "João;9383;75.5"
Use `split(";")` para separar nome, matrícula e nota.

04

04 — Validação de placa

Crie um método que receba uma placa e verifique se ela possui 7 caracteres após remover espaços.